Experiment 1: Exploratory Data using Pandas & NumPy

Objective: To perform exploratory data analysis (EDA) using Pandas and NumPy by generating a dataset, handling missing values, performing filtering and grouping operations, merging data, and visualizing relationships between features.

Code:

```
# ================================
# Lab Task 1: Exploratory Data (No CSV, Dataset in Code)
# ================================

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

# ================================
# Create Dataset in Code
# ================================
np.random.seed(42)

df = pd.DataFrame({
    "ID": np.arange(1, 51),
    "Category": np.random.choice(['A', 'B', 'C'], 50),
    "Feature1": np.random.randint(10, 100, 50).astype(float),
    "Feature2": np.random.randint(20, 120, 50).astype(float),
    "Feature3": np.random.randint(50, 200, 50).astype(float)
})

# Introduce missing values
for col in ["Feature1", "Feature2", "Feature3"]:
    df.loc[np.random.choice(df.index, 5, replace=False), col] = np.nan

# ================================
# Task 1: Basic Info
# ================================
print("===== First 5 Rows =====")
print(df.head())

print("\n===== Missing Values =====")
print(df.isnull().sum())

print("\n===== Summary Statistics =====")
print(df.describe())

# ================================
# Task 2: Fill Missing Values (Median)
# ================================
df_filled = df.copy()

for col in df_filled.select_dtypes(include=np.number).columns:
    df_filled[col].fillna(df_filled[col].median(), inplace=True)

print("\n===== After Filling Missing Values =====")
print(df_filled.isnull().sum())

# ================================
# Task 3: Filtering
# ================================
filtered_df = df_filled[df_filled["Feature1"] > 50]

print("\n===== Filtered Data (Feature1 > 50) =====")
print(filtered_df.head())

# ================================
# Task 4: Group By
# ================================
grouped = df_filled.groupby("Category").agg(["mean", "std"])

print("\n===== Grouped Data =====")
print(grouped)

# ================================
# Task 5: Merge DataFrames
```

```
# ================================
df_extra = pd.DataFrame({
    "ID": df_filled["ID"],
    "ExtraFeature": np.random.randint(100, 200, len(df_filled))
})

merged_df = pd.merge(df_filled, df_extra, on="ID")

print("\n===== Merged Data =====")
print(merged_df.head())

# ================================
# Task 6: New Column
# ================================
merged_df["Total"] = merged_df["Feature1"] + merged_df["Feature2"]

print("\n===== Data with New Column =====")
print(merged_df.head())

# ================================
# Task 7: Correlation Matrix
# ================================
corr = merged_df.corr(numeric_only=True)

print("\n===== Correlation Matrix =====")
print(corr)

sns.heatmap(corr, annot=True)
plt.title("Correlation Matrix")
plt.show()

# ================================
# Task 8: Scatter Plot
# ================================
plt.scatter(merged_df["Feature1"], merged_df["Feature2"])
plt.xlabel("Feature1")
plt.ylabel("Feature2")
plt.title("Feature1 vs Feature2")
plt.show()

# ================================
# Final Analysis
# ================================
print("\n===== Analysis =====")
print("Missing values handled using median.")
print("Filtering shows higher value samples.")
print("Grouping reveals category-wise trends.")
print("Correlation shows relationships between features.")
print("Scatter plot visualizes relationship between variables.")
```

Conclusion:

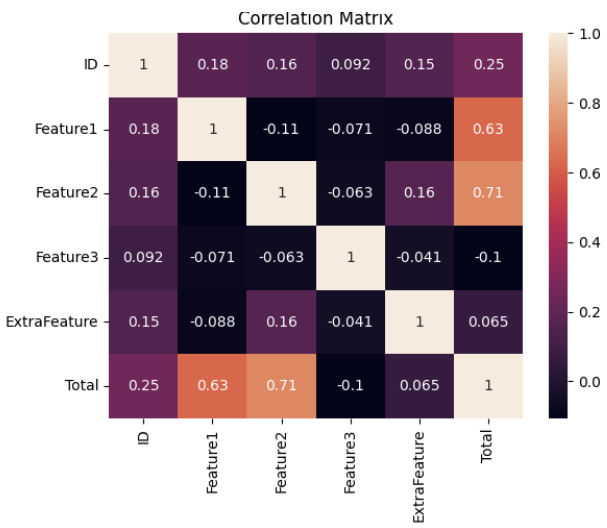
The experiment demonstrated the use of Pandas and NumPy for data analysis. Missing values were handled effectively, and operations like filtering, grouping, and merging provided useful insights. Visualizations helped understand relationships between features.

Output:

```
===== First 5 Rows =====
   ID Category  Feature1  Feature2  Feature3
0   1        C      18.0      90.0     145.0
1   2        A      99.0      28.0       NaN
2   3        C      62.0     107.0     145.0
3   4        C      11.0      20.0     181.0
4   5        A      93.0      27.0     192.0

===== Missing Values =====
ID          0
Category    0
Feature1    5
Feature2    5
Feature3    5
dtype: int64

===== Summary Statistics =====
              ID   Feature1    Feature2    Feature3
count  50.00000  45.000000   45.000000   45.000000
mean   25.50000  57.444444   67.311111  128.666667
std    14.57738  29.114629   32.054515   41.653331
min     1.00000  11.000000   20.000000   51.000000
25%    13.25000  33.000000   42.000000   94.000000
50%    25.50000  62.000000   63.000000  133.000000
75%    37.75000  83.000000   97.000000  162.000000
max    50.00000  99.000000  118.000000  192.000000
```

```
===== Summary Statistics =====
              ID   Feature1    Feature2    Feature3
count  50.00000  45.000000   45.000000   45.000000
mean   25.50000  57.444444   67.311111  128.666667
std    14.57738  29.114629   32.054515   41.653331
min     1.00000  11.000000   20.000000   51.000000
25%    13.25000  33.000000   42.000000   94.000000
50%    25.50000  62.000000   63.000000  133.000000
75%    37.75000  83.000000   97.000000  162.000000
max    50.00000  99.000000  118.000000  192.000000

===== After Filling Missing Values =====
ID          0
Category    0
Feature1    0
Feature2    0
Feature3    0
dtype: int64

===== Filtered Data (Feature1 > 50) =====
   ID Category  Feature1  Feature2  Feature3
1   2        A      99.0      28.0     133.0
2   3        C      62.0     107.0     145.0
4   5        A      93.0      27.0     192.0
5   6        A      69.0     107.0      78.0
6   7        C      80.0      82.0      85.0
```

```
===== Grouped Data =====
                ID              Feature1            Feature2            \
              mean        std       mean        std     mean        std
Category
A         25.333333  15.318834  66.333333  23.900588     63.6  28.276189
B         28.266667  13.079246  52.000000  32.215790     70.4  29.208120
C         23.550000  15.459540  56.000000  26.359858     66.7  33.879197

            Feature3
               mean        std
Category
A         120.533333  42.953241
B         128.533333  43.135777
C         135.950000  34.413087

===== Merged Data =====
   ID Category  Feature1  Feature2  Feature3  ExtraFeature
0   1        C      18.0      90.0     145.0           117
1   2        A      99.0      28.0     133.0           124
2   3        C      62.0     107.0     145.0           194
3   4        C      11.0      20.0     181.0           153
4   5        A      93.0      27.0     192.0           157

===== Data with New Column =====
   ID Category  Feature1  Feature2  Feature3  ExtraFeature  Total
0   1        C      18.0      90.0     145.0           117  108.0
1   2        A      99.0      28.0     133.0           124  127.0
2   3        C      62.0     107.0     145.0           194  169.0
3   4        C      11.0      20.0     181.0           153   31.0
4   5        A      93.0      27.0     192.0           157  120.0
```



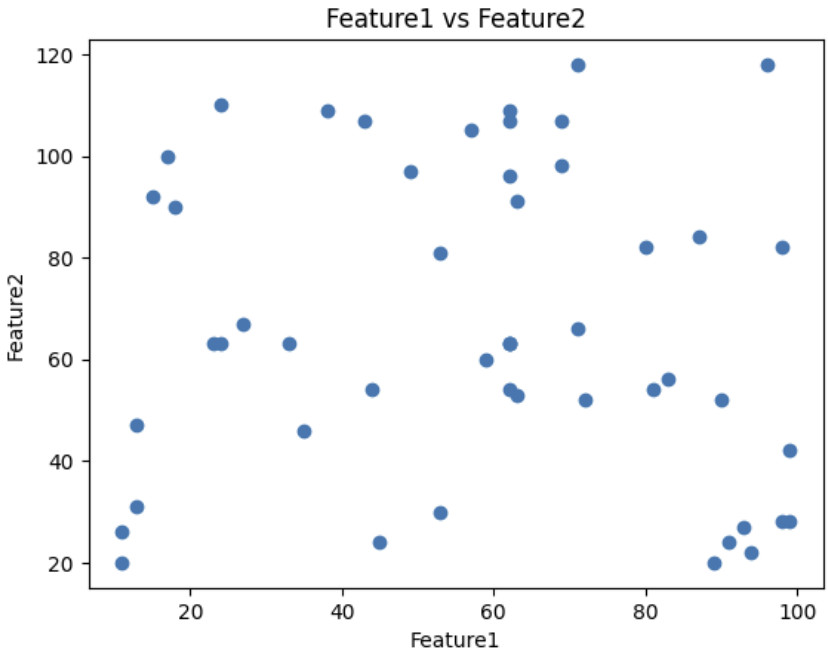


Figure 1: Outputs of Experiment 1

Experiment 2: Data Splitting & Evaluation

Objective: To understand data splitting using train-test split and cross-validation, and analyze consistency of feature distributions across datasets.

Code:

```
# ================================
# Lab Task 2: Data Splitting & Evaluation
# ================================

# 1. Import Libraries
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split, KFold

# ================================
# 2. Create Synthetic Dataset (500 rows)
# ================================
np.random.seed(42)

data = pd.DataFrame({
    "Feature1": np.random.normal(50, 10, 500),
    "Feature2": np.random.normal(30, 5, 500),
    "Feature3": np.random.uniform(0, 100, 500),
    "Feature4": np.random.normal(100, 20, 500)
})

print("===== Dataset Preview =====")
print(data.head())

print("\nShape:", data.shape)

# ================================
# 3. Data Types & Info
# ================================
print("\n===== Data Info =====")
print(data.info())

# ================================
# 4. Basic Statistics
# ================================
print("\n===== Statistical Summary =====")
print(data.describe())

# ================================
# 5. Histograms
# ================================
data.hist(figsize=(10, 8))
plt.suptitle("Feature Distributions (Histograms)")
plt.show()

# ================================
# 6. Boxplots
# ================================
plt.figure(figsize=(10, 6))
sns.boxplot(data=data)
plt.title("Boxplot of Features")
plt.show()

# ================================
# 7. Train-Test Split (70-30)
# ================================
train_data, val_data = train_test_split(data, test_size=0.3, random_state=42)

print("\nTrain Shape:", train_data.shape)
print("Validation Shape:", val_data.shape)

# ================================
# 8. Compare Statistics
# ================================
print("\n===== Train Statistics =====")
print(train_data.describe())
```

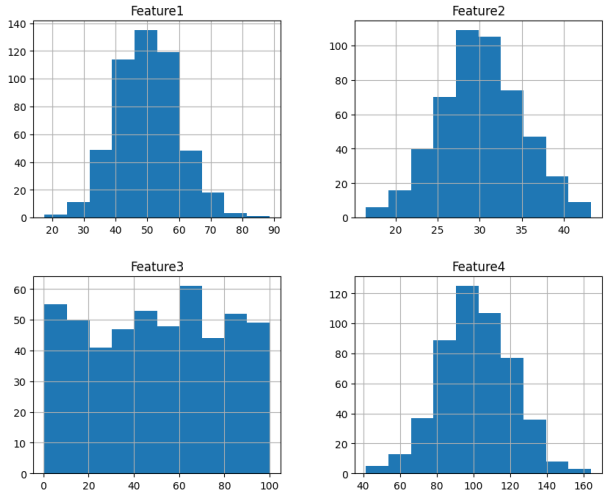
```
print("\n===== Validation Statistics =====")
print(val_data.describe())

# ================================
# 9. Overlapping Distribution Plots
# ================================
for col in data.columns:
    plt.figure()
    sns.kdeplot(train_data[col], label="Train", fill=True)
    sns.kdeplot(val_data[col], label="Validation", fill=True)
    plt.title(f"Distribution Comparison - {col}")
    plt.legend()
    plt.show()

# ================================
# 10. 5-Fold Cross Validation (Splitting Only)
# ================================
kf = KFold(n_splits=5, shuffle=True, random_state=42)

fold_stats = []

print("\n===== 5-Fold Splits =====")

for i, (train_idx, val_idx) in enumerate(kf.split(data)):
    fold_train = data.iloc[train_idx]
    fold_val = data.iloc[val_idx]

    print(f"\nFold {i+1}")
    print("Train size:", fold_train.shape, "Val size:", fold_val.shape)

    fold_stats.append(fold_train.mean())

# ================================
# 11. Analyze Variation Across Folds
# ================================
fold_stats_df = pd.DataFrame(fold_stats)

print("\n===== Mean Variation Across Folds =====")
print(fold_stats_df)

fold_stats_df.plot(kind='bar')
plt.title("Feature Mean Variation Across Folds")
plt.xlabel("Fold")
plt.ylabel("Mean Value")
plt.xticks(rotation=0)
plt.show()

# ================================
# 12. Final Analysis
# ================================
print("\n===== Analysis =====")

print("1. Hold-out Method (70-30 split):")
print("- Simple and fast.")
print("- May suffer from high variance depending on split.")

print("\n2. Cross-Validation:")
print("- Uses multiple splits     more reliable evaluation.")
print("- Reduces variance in performance estimates.")

print("\n3. Observations:")
print("- Feature distributions are similar across train and validation.")
print("- Cross-validation shows stable statistics across folds.")

print("\nConclusion:")
print("- Cross-validation is more robust than a single hold-out split.")
```

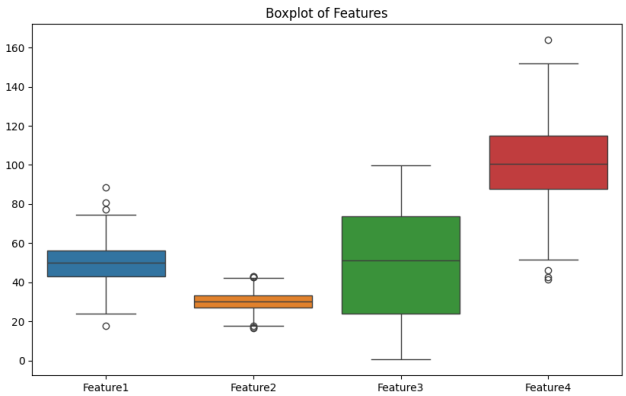
Conclusion:

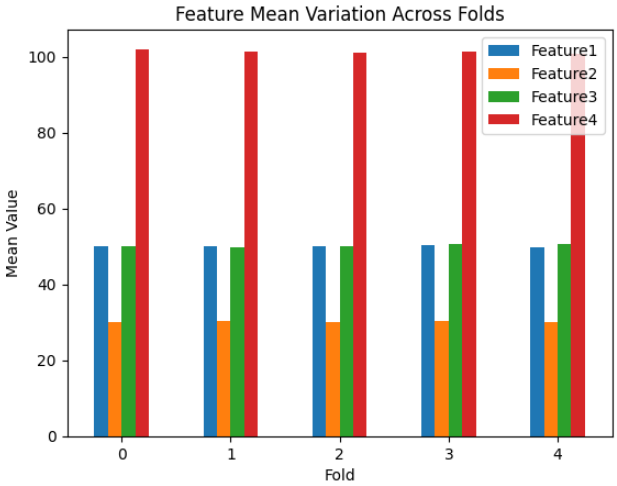
The experiment demonstrated both hold-out and cross-validation methods. Train-test split provided a simple evaluation, while cross-validation gave more stable results. Feature distributions remained consistent, showing reliable data splitting.

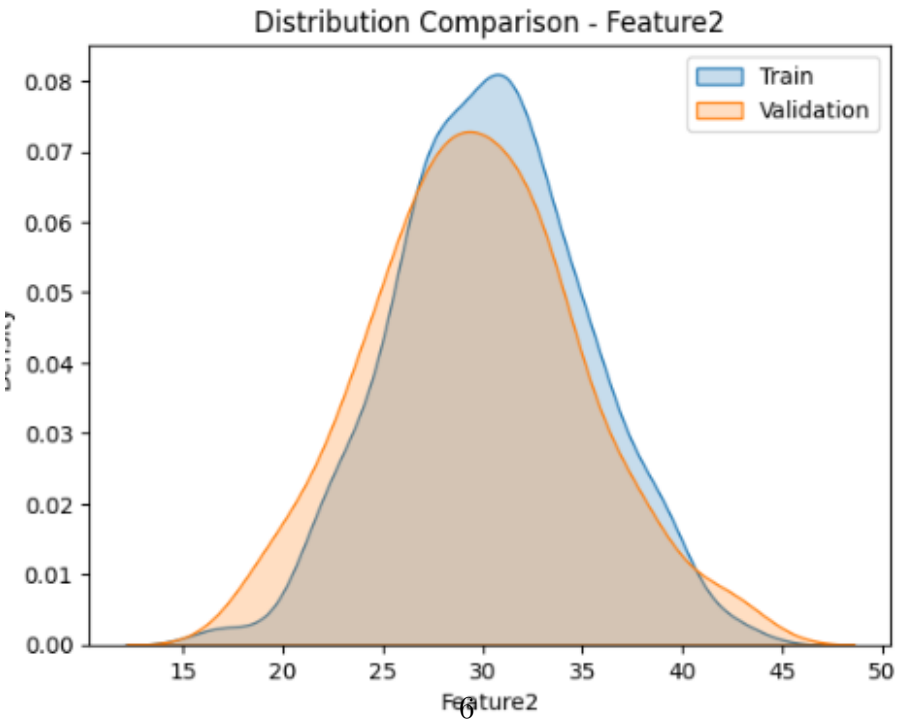
Output:

```
===== Dataset Preview =====
    Feature1   Feature2   Feature3    Feature4
0  54.967142  34.630888  16.748258   92.693569
1  48.617357  39.547083  10.456784  103.693606
2  56.476885  23.007162  63.643025   73.057474
3  65.230299  32.814846  70.647573   80.567719
4  47.658466  26.746787   3.158614  124.008278

Shape: (500, 4)

===== Data Info =====
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 500 entries, 0 to 499
Data columns (total 4 columns):
 #   Column    Non-Null Count  Dtype
---  ------    --------------  -----
 0   Feature1  500 non-null    float64
 1   Feature2  500 non-null    float64
 2   Feature3  500 non-null    float64
 3   Feature4  500 non-null    float64
dtypes: float64(4)
memory usage: 15.8 KB
None

===== Statistical Summary =====
         Feature1    Feature2    Feature3    Feature4
count  500.000000  500.000000  500.000000  500.000000
mean    50.068380   30.159131   50.217981  101.250729
std      9.812532    4.889986   28.960865   19.630581
min     17.587327   16.515567    0.493998   41.572990
25%     42.996926   27.023541   23.877111   87.833756
50%     50.127971   30.142658   51.082202  100.455311
75%     56.367833   33.256211   73.651995  114.792795
max     88.527315   43.161910   99.941373  163.862151
```









Distribution Comparison - Feature3

Experiment 3: Performance Measures & Linear Regression (Scratch)

Objective: To compute classification performance metrics using a confusion matrix and implement linear regression from scratch using the normal equation.

Code:

```
# ================================
# Lab Task 3: Performance Measures + Linear Regression (Scratch)
# ================================

# 1. Import Libraries
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

# ================================
# PART 1: PERFORMANCE MEASURES
# ================================

# Given Data
y_actual = np.array([1, 0, 1, 0, 1, 0, 1, 0])
y_pred = np.array([1, 1, 1, 0, 0, 0, 1, 0])

# Calculate TP, TN, FP, FN
TP = np.sum((y_actual == 1) & (y_pred == 1))
TN = np.sum((y_actual == 0) & (y_pred == 0))
FP = np.sum((y_actual == 0) & (y_pred == 1))
FN = np.sum((y_actual == 1) & (y_pred == 0))

print("===== Confusion Matrix Values =====")
print("TP:", TP, "TN:", TN, "FP:", FP, "FN:", FN)

# Confusion Matrix
cm = np.array([[TP, FN],
               [FP, TN]])

print("\nConfusion Matrix:\n", cm)

sns.heatmap(cm, annot=True, fmt='d', xticklabels=['Pred 1','Pred 0'], yticklabels=['Actual 1','
    Actual 0'])
plt.title("Confusion Matrix")
plt.show()

# Metrics
accuracy = (TP + TN) / (TP + TN + FP + FN)
precision = TP / (TP + FP)
recall = TP / (TP + FN)
f1 = 2 * (precision * recall) / (precision + recall)
specificity = TN / (TN + FP)

print("\n===== Metrics =====")
print("Accuracy:", accuracy)
print("Precision:", precision)
print("Recall:", recall)
print("F1 Score:", f1)
print("Specificity:", specificity)

# Interpretation
print("\n===== Interpretation =====")
print("Accuracy: Overall correctness of the model.")
print("Precision: Out of predicted positives, how many are correct.")
print("Recall: Out of actual positives, how many are detected.")
print("F1 Score: Balance between precision and recall.")
print("Specificity: Ability to correctly identify negatives.")

print("\nReal-life case where Recall is more important:")
print("Disease detection (e.g., cancer screening).")

# ================================
# PART 2: LINEAR REGRESSION FROM SCRATCH
# ================================
```

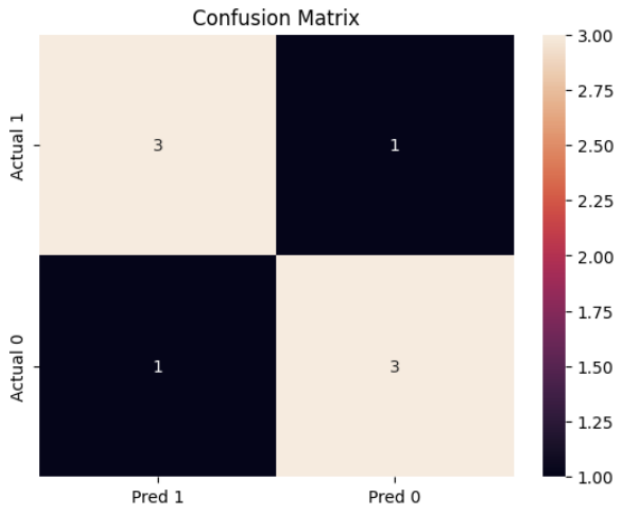
```
# Create Dataset
np.random.seed(42)
X = 2 * np.random.rand(100, 1)
y = 4 + 3 * X + np.random.randn(100, 1)

# Train-Test Split
split = int(0.8 * len(X))
X_train, X_test = X[:split], X[split:]
y_train, y_test = y[:split], y[split:]

# Add Bias Term
X_train_b = np.c_[np.ones((X_train.shape[0], 1)), X_train]
X_test_b = np.c_[np.ones((X_test.shape[0], 1)), X_test]

# Normal Equation
theta = np.linalg.inv(X_train_b.T @ X_train_b) @ X_train_b.T @ y_train

# Predictions
y_pred_lr = X_test_b @ theta

# Evaluation
mse = np.mean((y_test - y_pred_lr) ** 2)
ss_total = np.sum((y_test - np.mean(y_test)) ** 2)
ss_residual = np.sum((y_test - y_pred_lr) ** 2)
r2 = 1 - (ss_residual / ss_total)

print("\n===== Linear Regression Results =====")
print("MSE:", mse)
print("R2 Score:", r2)

# Plot
plt.scatter(X_test, y_test, label="Actual")
plt.plot(X_test, y_pred_lr, color='red', label="Predicted Line")
plt.title("Linear Regression from Scratch")
plt.xlabel("X")
plt.ylabel("y")
plt.legend()
plt.show()

# ================================
# FINAL ANALYSIS
# ================================
print("\n===== Final Analysis =====")
print("Performance metrics show classification quality.")
print("Linear regression successfully models linear relationship.")
print("Low MSE and good R2 indicate good fit.")
```

Conclusion:

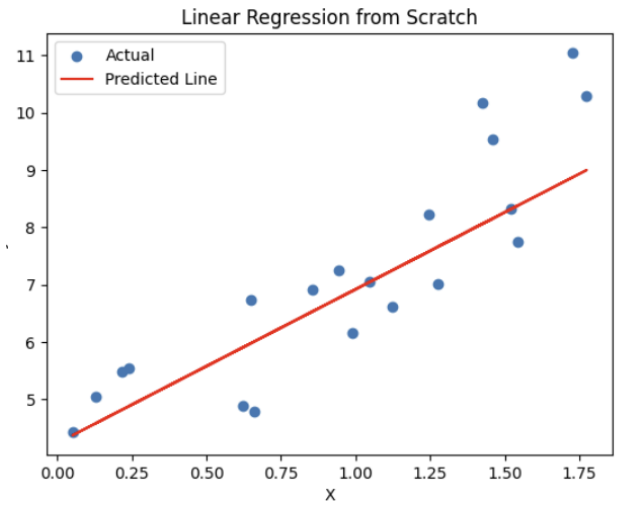
The experiment demonstrated evaluation of classification models using metrics such as accuracy, precision, recall, and F1 score. Linear regression was implemented from scratch and showed a good fit with low error and high R2 score.

Output:

```
===== Confusion Matrix Values =====
TP: 3 TN: 3 FP: 1 FN: 1

Confusion Matrix:
 [[3 1]
 [1 3]]
```




```
===== Metrics =====
Accuracy: 0.75
Precision: 0.75
Recall: 0.75
F1 Score: 0.75
Specificity: 0.75

===== Interpretation =====
Accuracy: Overall correctness of the model.
Precision: Out of predicted positives, how many are correct.
Recall: Out of actual positives, how many are detected.
F1 Score: Balance between precision and recall.
Specificity: Ability to correctly identify negatives.

Real-life case where Recall is more important:
Disease detection (e.g., cancer screening).

===== Linear Regression Results =====
MSE: 0.9755437477937203
R2 Score: 0.7296738412715471
```




```
===== Final Analysis =====
Performance metrics show classification quality.
Linear regression successfully models linear relationship.
Low MSE and good R2 indicate good fit.
```

Figure 3: Outputs of Experiment 3

Experiment 4: Linear Regression (Math, Library, GD & Regularization)

Objective: To implement linear regression using mathematical formulation, library methods, and gradient descent, and analyze the effect of L2 regularization on model performance and convergence.

Code:

```
# ================================
# 1. Import Libraries
# ================================
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.datasets import load_diabetes
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import mean_squared_error, r2_score
from sklearn.linear_model import LinearRegression

# ================================
# 2. Load Dataset
# ================================
data = load_diabetes()
X = data.data
y = data.target

# ================================
# 3. Train-Test Split
# ================================
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# ================================
# 4. Feature Scaling
# ================================
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# ================================
# 5. Linear Regression (Mathematical - Normal Equation)
# ================================
X_train_bias = np.c_[np.ones(X_train.shape[0]), X_train]
X_test_bias = np.c_[np.ones(X_test.shape[0]), X_test]

theta = np.linalg.inv(X_train_bias.T @ X_train_bias) @ X_train_bias.T @ y_train

y_pred_math = X_test_bias @ theta

mse_math = mean_squared_error(y_test, y_pred_math)
r2_math = r2_score(y_test, y_pred_math)

print("\n===== Linear Regression (Mathematical) =====")
print("MSE:", mse_math)
print("R2 Score:", r2_math)

# ================================
# 6. Linear Regression (Library)
# ================================
model_lib = LinearRegression()
model_lib.fit(X_train, y_train)

y_pred_lib = model_lib.predict(X_test)

mse_lib = mean_squared_error(y_test, y_pred_lib)
r2_lib = r2_score(y_test, y_pred_lib)

print("\n===== Linear Regression (Library) =====")
print("MSE:", mse_lib)
```

```
print("R2 Score:", r2_lib)

# ================================
# 7. Gradient Descent Implementation
# ================================
class LinearRegressionGD:
    def __init__(self, lr=0.01, epochs=1000, l2=0):
        self.lr = lr
        self.epochs = epochs
        self.l2 = l2

    def fit(self, X, y):
        self.m, self.n = X.shape
        self.w = np.zeros(self.n)
        self.b = 0
        self.losses = []

        for _ in range(self.epochs):
            y_pred = np.dot(X, self.w) + self.b

            error = y_pred - y
            loss = np.mean(error ** 2)
            self.losses.append(loss)

            dw = (2/self.m) * np.dot(X.T, error) + self.l2 * self.w
            db = (2/self.m) * np.sum(error)

            self.w -= self.lr * dw
            self.b -= self.lr * db

    def predict(self, X):
        return np.dot(X, self.w) + self.b

# ================================
# 8. Train GD Model (No Regularization)
# ================================
gd_model = LinearRegressionGD(lr=0.01, epochs=1000, l2=0)
gd_model.fit(X_train, y_train)

y_pred_gd = gd_model.predict(X_test)

mse_gd = mean_squared_error(y_test, y_pred_gd)
r2_gd = r2_score(y_test, y_pred_gd)

print("\n===== Gradient Descent (No Regularization) =====")
print("MSE:", mse_gd)
print("R2 Score:", r2_gd)

# ================================
# 9. Plot Loss Curve
# ================================
plt.plot(gd_model.losses)
plt.title("Loss vs Iterations (No Regularization)")
plt.xlabel("Iterations")
plt.ylabel("MSE Loss")
plt.show()

# ================================
# 10. GD with L2 Regularization
# ================================
gd_l2_model = LinearRegressionGD(lr=0.01, epochs=1000, l2=0.1)
gd_l2_model.fit(X_train, y_train)

y_pred_l2 = gd_l2_model.predict(X_test)

mse_l2 = mean_squared_error(y_test, y_pred_l2)
r2_l2 = r2_score(y_test, y_pred_l2)

print("\n===== Gradient Descent (L2 Regularization) =====")
print("MSE:", mse_l2)
print("R2 Score:", r2_l2)

# ================================
# 11. Plot Loss Comparison
# ================================
plt.plot(gd_model.losses, label="No Regularization")
plt.plot(gd_l2_model.losses, label="L2 Regularization")
plt.title("Loss Comparison")
```

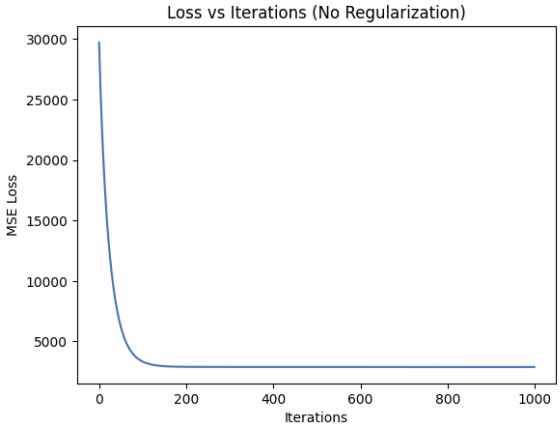
```
plt.xlabel("Iterations")
plt.ylabel("MSE Loss")
plt.legend()
plt.show()

# ================================
# 12. Model Comparison Table
# ================================
comparison = pd.DataFrame({
    "Model": ["Math", "Library", "GD", "GD + L2"],
    "MSE": [mse_math, mse_lib, mse_gd, mse_l2],
    "R2 Score": [r2_math, r2_lib, r2_gd, r2_l2]
})

print("\n===== Model Comparison =====")
print(comparison)

comparison.set_index("Model").plot(kind='bar')
plt.title("Model Comparison")
plt.xticks(rotation=0)
plt.show()

# ================================
# 13. Coefficient Stability
# ================================
coeff_df = pd.DataFrame({
    "GD": gd_model.w,
    "GD + L2": gd_l2_model.w
})

print("\n===== Coefficient Comparison =====")
print(coeff_df)

coeff_df.plot(kind='bar')
plt.title("Coefficient Stability (GD vs L2)")
plt.show()

# ================================
# 14. Final Analysis
# ================================
print("\n===== Analysis =====")

print("1. Mathematical vs Library:")
print("- Both give similar results (closed-form solution).")

print("\n2. Gradient Descent:")
print("- Converges gradually toward optimal solution.")
print("- Loss decreases smoothly over iterations.")

print("\n3. L2 Regularization:")
print("- Reduces magnitude of coefficients.")
print("- Helps prevent overfitting.")
print("- Slight increase in bias but improves stability.")

print("\n4. Convergence Behavior:")
print("- Proper learning rate ensures smooth convergence.")
print("- Too high LR would cause divergence.")

print("\n5. Conclusion:")
print("- Normal equation is fast for small datasets.")
print("- GD is scalable for large datasets.")
print("- Regularization improves generalization.")
```

Conclusion:

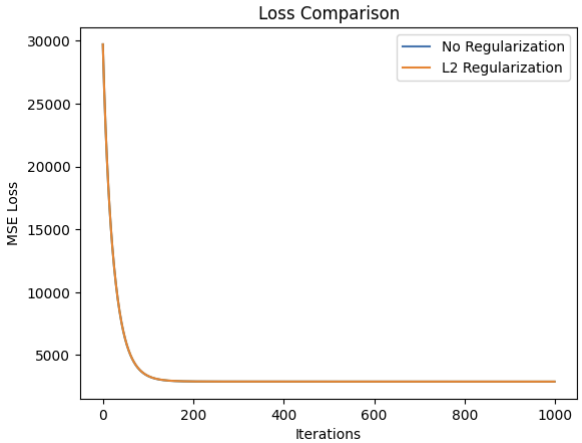
The experiment compared multiple approaches to linear regression. The mathematical and library methods produced similar results, while gradient descent showed gradual convergence. L2 regularization improved model stability by reducing coefficient magnitude and helping prevent overfitting.

Output:

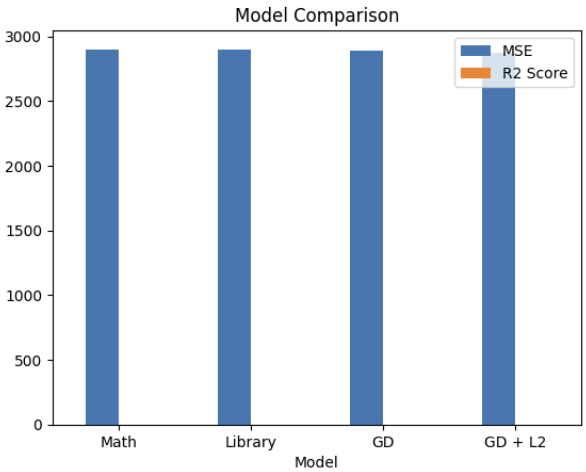
```
===== Linear Regression (Mathematical) =====
MSE: 2900.193628493486
R2 Score: 0.4526027629719186

===== Linear Regression (Library) =====
MSE: 2900.193628934823
R2 Score: 0.45260276297191926

===== Gradient Descent (No Regularization) =====
MSE: 2885.6135403538306
R2 Score: 0.45535468266612844
```



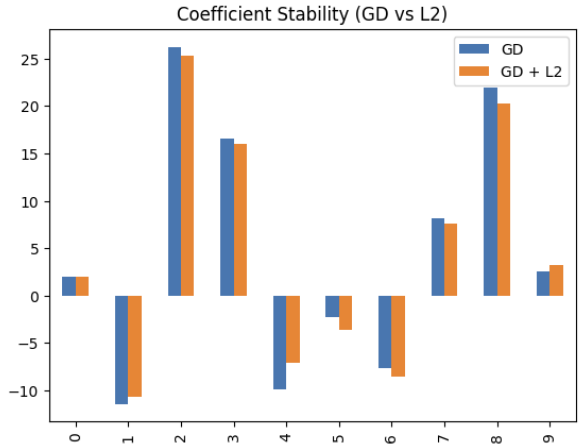

```
===== Gradient Descent (L2 Regularization) =====
MSE: 2869.603829621242
R2 Score: 0.4583764365706049
```




```
===== Model Comparison =====
     Model          MSE  R2 Score
0     Math  2900.193628  0.452603
1  Library  2900.193628  0.452603
2       GD  2885.613540  0.455355
3  GD + L2  2869.603830  0.458376
```




```
===== Coefficient Comparison =====
          GD    GD + L2
0   1.937195   1.969186
1 -11.432570 -10.713248
2  26.267164  25.275866
3  16.607099  16.028000
4  -9.879779  -7.095851
5  -2.304101  -3.639934
6  -7.683675  -8.613120
7   8.201066   7.612960
8  21.950879  20.275721
9   2.602733   3.190681
```




```
===== Analysis =====
1. Mathematical vs Library:
- Both give similar results (closed-form solution).

2. Gradient Descent:
- Converges gradually toward optimal solution.
- Loss decreases smoothly over iterations.

3. L2 Regularization:
- Reduces magnitude of coefficients.
- Helps prevent overfitting.
- Slight increase in bias but improves stability.

4. Convergence Behavior:
- Proper learning rate ensures smooth convergence.
- Too high LR would cause divergence.

5. Conclusion:
- Normal equation is fast for small datasets.
- GD is scalable for large datasets.
```

Experiment 5: Logistic Regression (Binary, GD & Multinomial)

Objective: To implement logistic regression for binary and multi-class classification using both library and gradient descent approaches, and evaluate performance using metrics such as accuracy, precision, recall, F1 score, ROC curve, and AUC.

Code:

```
# ================================
# 1. Import Libraries
# ================================
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.datasets import load_breast_cancer, load_iris
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import (
    accuracy_score, precision_score, recall_score, f1_score,
    confusion_matrix, classification_report, roc_curve, auc
)
from sklearn.linear_model import LogisticRegression

# ================================
# 2. Load Binary Dataset (Breast Cancer)
# ================================
data = load_breast_cancer()
X = data.data
y = data.target

# ================================
# 3. Train-Test Split
# ================================
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# ================================
# 4. Feature Scaling
# ================================
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# ================================
# 5. Logistic Regression (Library)
# ================================
model_lib = LogisticRegression(max_iter=5000)
model_lib.fit(X_train, y_train)

y_pred_lib = model_lib.predict(X_test)
y_prob_lib = model_lib.predict_proba(X_test)[:, 1]

# ================================
# 6. Evaluation (Library Model)
# ================================
print("\n===== Logistic Regression (Library) =====")

acc_lib = accuracy_score(y_test, y_pred_lib)
prec_lib = precision_score(y_test, y_pred_lib)
rec_lib = recall_score(y_test, y_pred_lib)
f1_lib = f1_score(y_test, y_pred_lib)

print("Accuracy:", acc_lib)
print("Precision:", prec_lib)
print("Recall:", rec_lib)
print("F1 Score:", f1_lib)

cm_lib = confusion_matrix(y_test, y_pred_lib)
```

```
print("\nConfusion Matrix:\n", cm_lib)

print("\nClassification Report:\n", classification_report(y_test, y_pred_lib))

# Confusion Matrix Plot
sns.heatmap(cm_lib, annot=True, fmt='d')
plt.title("Confusion Matrix - Library Logistic Regression")
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.show()

# ================================
# 7. ROC Curve & AUC
# ================================
fpr, tpr, _ = roc_curve(y_test, y_prob_lib)
roc_auc = auc(fpr, tpr)

plt.plot(fpr, tpr, label="AUC = %.3f" % roc_auc)
plt.plot([0, 1], [0, 1], linestyle='--')
plt.title("ROC Curve - Library Logistic Regression")
plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.legend()
plt.show()

print("AUC Score:", roc_auc)

# ================================
# 8. Logistic Regression using Gradient Descent
# ================================
class LogisticRegressionGD:
    def __init__(self, lr=0.01, epochs=1000):
        self.lr = lr
        self.epochs = epochs

    def sigmoid(self, z):
        return 1 / (1 + np.exp(-z))

    def fit(self, X, y):
        self.m, self.n = X.shape
        self.w = np.zeros(self.n)
        self.b = 0

        for _ in range(self.epochs):
            linear = np.dot(X, self.w) + self.b
            y_pred = self.sigmoid(linear)

            dw = (1 / self.m) * np.dot(X.T, (y_pred - y))
            db = (1 / self.m) * np.sum(y_pred - y)

            self.w -= self.lr * dw
            self.b -= self.lr * db

    def predict(self, X):
        linear = np.dot(X, self.w) + self.b
        y_pred = self.sigmoid(linear)
        return [1 if i > 0.5 else 0 for i in y_pred]

    def predict_proba(self, X):
        linear = np.dot(X, self.w) + self.b
        return self.sigmoid(linear)

# Train GD model
model_gd = LogisticRegressionGD(lr=0.01, epochs=2000)
model_gd.fit(X_train, y_train)

y_pred_gd = model_gd.predict(X_test)
y_prob_gd = model_gd.predict_proba(X_test)

# ================================
# 9. Evaluation (GD Model)
# ================================
print("\n===== Logistic Regression (Gradient Descent) =====")

acc_gd = accuracy_score(y_test, y_pred_gd)
prec_gd = precision_score(y_test, y_pred_gd)
rec_gd = recall_score(y_test, y_pred_gd)
f1_gd = f1_score(y_test, y_pred_gd)
```

```
print("Accuracy:", acc_gd)
print("Precision:", prec_gd)
print("Recall:", rec_gd)
print("F1 Score:", f1_gd)

cm_gd = confusion_matrix(y_test, y_pred_gd)
print("\nConfusion Matrix:\n", cm_gd)

sns.heatmap(cm_gd, annot=True, fmt='d')
plt.title("Confusion Matrix - Gradient Descent")
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.show()

# ROC for GD
fpr_gd, tpr_gd, _ = roc_curve(y_test, y_prob_gd)
auc_gd = auc(fpr_gd, tpr_gd)

plt.plot(fpr_gd, tpr_gd, label="GD AUC = %.3f" % auc_gd)
plt.plot([0, 1], [0, 1], linestyle='--')
plt.title("ROC Curve - Gradient Descent")
plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.legend()
plt.show()

print("AUC Score (GD):", auc_gd)

# ================================
# 10. Comparison Table (Binary)
# ================================
comparison_binary = pd.DataFrame({
    "Model": ["Library", "Gradient Descent"],
    "Accuracy": [acc_lib, acc_gd],
    "Precision": [prec_lib, prec_gd],
    "Recall": [rec_lib, rec_gd],
    "F1 Score": [f1_lib, f1_gd],
    "AUC": [roc_auc, auc_gd]
})

print("\n===== Binary Model Comparison =====")
print(comparison_binary)

comparison_binary.set_index("Model").plot(kind='bar')
plt.title("Binary Model Comparison")
plt.xticks(rotation=0)
plt.show()

print("\nAnalysis: Library model usually performs slightly better due to optimized solvers.")

# ================================
# 11. Multinomial Logistic Regression (Iris)
# ================================
iris = load_iris()
X_multi = iris.data
y_multi = iris.target

X_train_m, X_test_m, y_train_m, y_test_m = train_test_split(
    X_multi, y_multi, test_size=0.2, random_state=42
)

scaler_m = StandardScaler()
X_train_m = scaler_m.fit_transform(X_train_m)
X_test_m = scaler_m.transform(X_test_m)

model_multi = LogisticRegression(multi_class='multinomial', max_iter=5000)
model_multi.fit(X_train_m, y_train_m)

y_pred_multi = model_multi.predict(X_test_m)

# ================================
# 12. Evaluation (Multinomial)
# ================================
print("\n===== Multinomial Logistic Regression =====")

acc_multi = accuracy_score(y_test_m, y_pred_multi)
print("Accuracy:", acc_multi)
```

```
cm_multi = confusion_matrix(y_test_m, y_pred_multi)
print("\nConfusion Matrix:\n", cm_multi)

sns.heatmap(cm_multi, annot=True, fmt='d')
plt.title("Confusion Matrix - Multinomial")
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.show()

# ================================
# 13. Final Comparison
# ================================
final_comparison = pd.DataFrame({
    "Model": ["Binary (Library)", "Binary (GD)", "Multinomial"],
    "Accuracy": [acc_lib, acc_gd, acc_multi]
})

print("\n===== Final Comparison =====")
print(final_comparison)

final_comparison.set_index("Model").plot(kind='bar')
plt.title("Final Model Comparison")
plt.xticks(rotation=0)
plt.show()

print("\nFinal Analysis:")
print("- Binary Logistic Regression is suitable for 2-class problems.")
print("- Multinomial Logistic Regression handles multi-class classification effectively.")
print("- Gradient Descent implementation works but is less optimized than sklearn.")
```

Conclusion:

The experiment demonstrated logistic regression for both binary and multi-class classification. The library model achieved slightly better performance due to optimized solvers, while the gradient descent implementation showed correct learning behavior. ROC and AUC provided deeper insights into model performance, and multinomial logistic regression effectively handled multi-class classification.

Output:

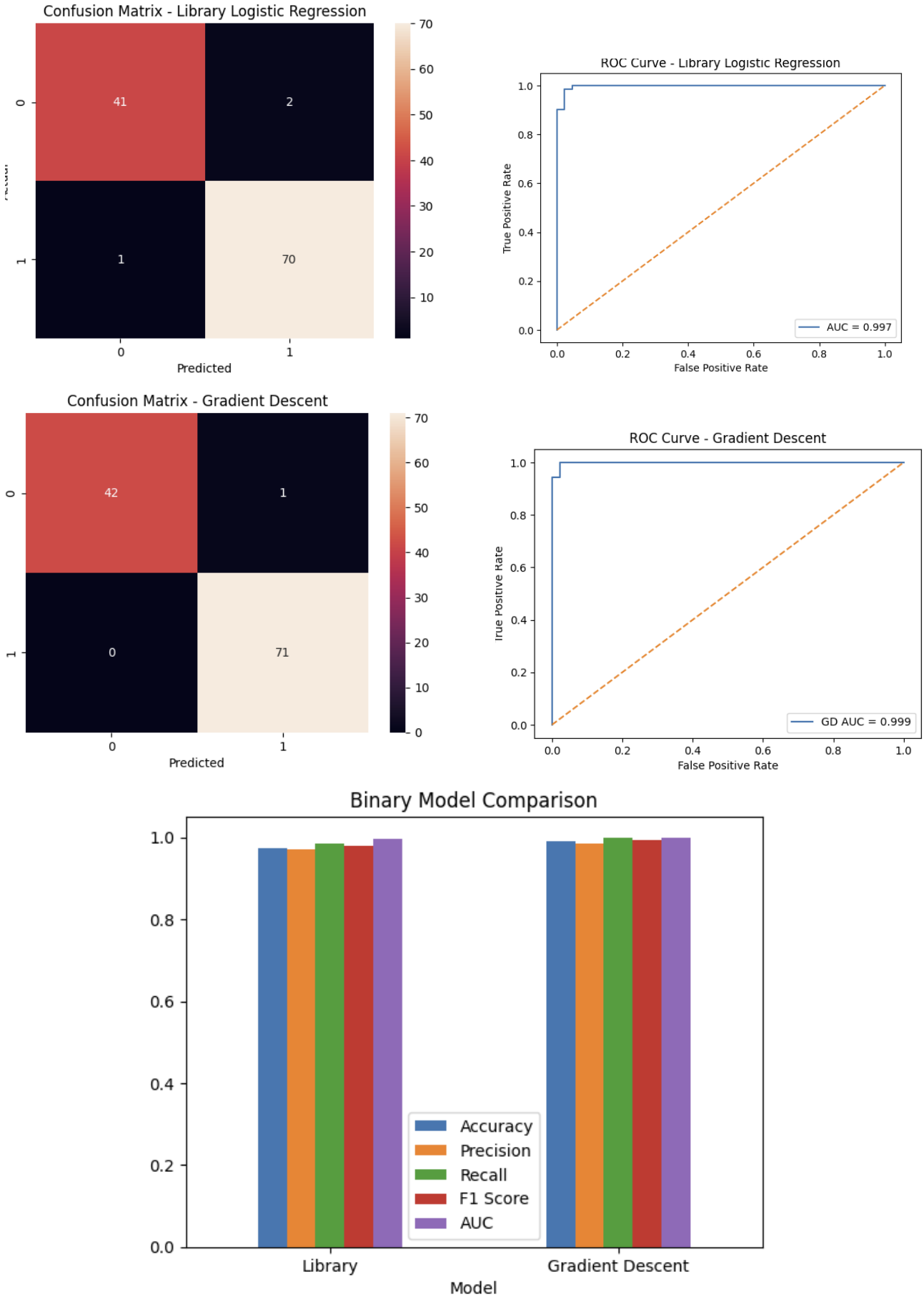


Figure 5: Outputs of Experiment 5

Experiment 6: Naïve Bayes and Support Vector Machine

Objective: To implement and compare Naïve Bayes and Support Vector Machine (SVM) models using different kernels, and evaluate their performance using classification metrics and ROC curves.

Code:

```
# ================================
# Lab Task 6: Na ve Bayes and SVM
# ================================

# 1. Import Libraries
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.datasets import load_wine
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, label_binarize
from sklearn.naive_bayes import GaussianNB
from sklearn.svm import SVC
from sklearn.metrics import (
    confusion_matrix, accuracy_score, precision_score,
    recall_score, f1_score, classification_report,
    roc_curve, auc
)

# ================================
# 2. Load Dataset
# ================================
data = load_wine()
X = data.data
y = data.target

print("Feature Names:\n", data.feature_names)
print("\nTarget Classes:\n", data.target_names)

# ================================
# 3. Train-Test Split (70-30)
# ================================
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42, stratify=y
)

# Scaling
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# ================================
# 4. Gaussian Na ve Bayes
# ================================
nb_model = GaussianNB()
nb_model.fit(X_train, y_train)

y_pred_nb = nb_model.predict(X_test)

# ================================
# 5. Evaluation (Na ve Bayes)
# ================================
print("\n===== Na ve Bayes =====")

acc_nb = accuracy_score(y_test, y_pred_nb)
prec_nb = precision_score(y_test, y_pred_nb, average='weighted')
rec_nb = recall_score(y_test, y_pred_nb, average='weighted')
f1_nb = f1_score(y_test, y_pred_nb, average='weighted')

print("Accuracy:", acc_nb)
print("Precision:", prec_nb)
print("Recall:", rec_nb)
print("F1 Score:", f1_nb)

cm_nb = confusion_matrix(y_test, y_pred_nb)
```

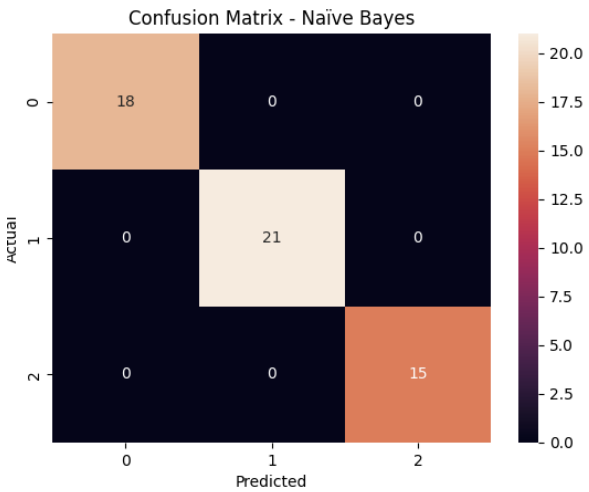
```
print("\nConfusion Matrix:\n", cm_nb)

sns.heatmap(cm_nb, annot=True, fmt='d')
plt.title("Confusion Matrix - Na ve Bayes")
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.show()

print("\nClassification Report:\n", classification_report(y_test, y_pred_nb))

# ================================
# 6. ROC Curve (Multiclass - One vs Rest)
# ================================
y_test_bin = label_binarize(y_test, classes=np.unique(y))
y_prob_nb = nb_model.predict_proba(X_test)

for i in range(y_test_bin.shape[1]):
    fpr, tpr, _ = roc_curve(y_test_bin[:, i], y_prob_nb[:, i])
    roc_auc = auc(fpr, tpr)
    plt.plot(fpr, tpr, label=f"Class {i} AUC = {roc_auc:.2f}")

plt.plot([0, 1], [0, 1], linestyle='--')
plt.title("ROC Curve - Na ve Bayes")
plt.xlabel("FPR")
plt.ylabel("TPR")
plt.legend()
plt.show()

# ================================
# 7. SVM (Linear Kernel)
# ================================
svm_linear = SVC(kernel='linear', probability=True)
svm_linear.fit(X_train, y_train)

y_pred_lin = svm_linear.predict(X_test)

# ================================
# 8. Evaluation (SVM Linear)
# ================================
print("\n===== SVM (Linear Kernel) =====")

acc_lin = accuracy_score(y_test, y_pred_lin)
prec_lin = precision_score(y_test, y_pred_lin, average='weighted')
rec_lin = recall_score(y_test, y_pred_lin, average='weighted')
f1_lin = f1_score(y_test, y_pred_lin, average='weighted')

print("Accuracy:", acc_lin)
print("Precision:", prec_lin)
print("Recall:", rec_lin)
print("F1 Score:", f1_lin)

cm_lin = confusion_matrix(y_test, y_pred_lin)

sns.heatmap(cm_lin, annot=True, fmt='d')
plt.title("Confusion Matrix - SVM Linear")
plt.show()

# ================================
# 9. SVM (RBF Kernel)
# ================================
svm_rbf = SVC(kernel='rbf', probability=True)
svm_rbf.fit(X_train, y_train)

y_pred_rbf = svm_rbf.predict(X_test)

# ================================
# 10. Evaluation (SVM RBF)
# ================================
print("\n===== SVM (RBF Kernel) =====")

acc_rbf = accuracy_score(y_test, y_pred_rbf)
prec_rbf = precision_score(y_test, y_pred_rbf, average='weighted')
rec_rbf = recall_score(y_test, y_pred_rbf, average='weighted')
f1_rbf = f1_score(y_test, y_pred_rbf, average='weighted')

print("Accuracy:", acc_rbf)
print("Precision:", prec_rbf)
print("Recall:", rec_rbf)
```

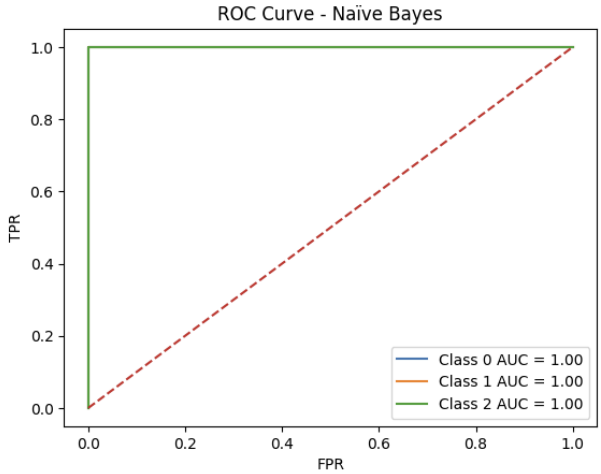
```
print("F1 Score:", f1_rbf)

cm_rbf = confusion_matrix(y_test, y_pred_rbf)

sns.heatmap(cm_rbf, annot=True, fmt='d')
plt.title("Confusion Matrix - SVM RBF")
plt.show()

# ================================
# 11. Comparison Table
# ================================
comparison = pd.DataFrame({
    "Model": ["Na ve Bayes", "SVM Linear", "SVM RBF"],
    "Accuracy": [acc_nb, acc_lin, acc_rbf],
    "Precision": [prec_nb, prec_lin, prec_rbf],
    "Recall": [rec_nb, rec_lin, rec_rbf],
    "F1 Score": [f1_nb, f1_lin, f1_rbf]
})

print("\n===== Model Comparison =====")
print(comparison)

comparison.set_index("Model").plot(kind='bar')
plt.title("Model Comparison")
plt.xticks(rotation=0)
plt.show()

# ================================
# 12. Analysis
# ================================
print("\n===== Analysis =====")

print("1. Na ve Bayes:")
print("- Assumes feature independence.")
print("- Works fast but may be less accurate.")

print("\n2. SVM Linear:")
print("- Works well for linearly separable data.")
print("- Good generalization.")

print("\n3. SVM RBF:")
print("- Handles non-linear boundaries.")
print("- Usually gives best performance.")

print("\n4. Kernel Effect:")
print("- Linear kernel is simpler but limited.")
print("- RBF kernel captures complex relationships.")

print("\nConclusion:")
print("- SVM (RBF) typically performs best on Wine dataset.")
```

Conclusion:

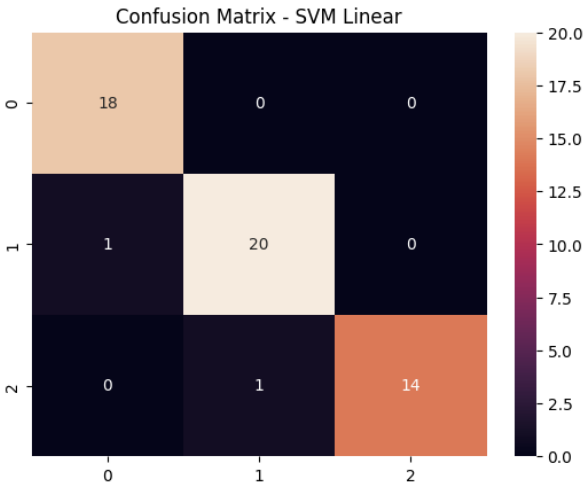
The experiment compared Naïve Bayes and SVM models on a multi-class dataset. Naïve Bayes provided fast predictions with reasonable accuracy, while SVM models performed better, especially with the RBF kernel. The results show that kernel choice significantly impacts SVM performance, with RBF handling non-linear patterns more effectively.

Output:



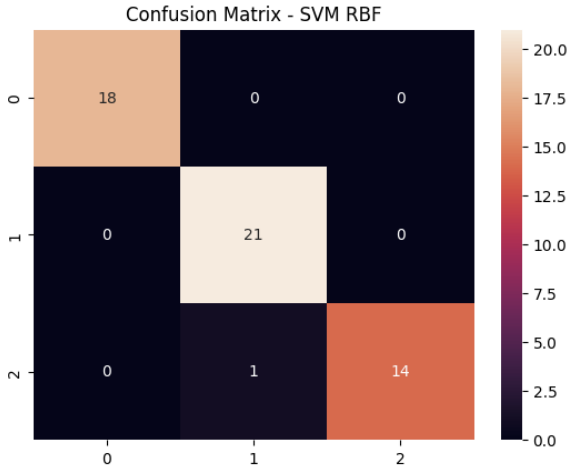



```
===== SVM (Linear Kernel) =====
Accuracy: 0.9629629629629629
Precision: 0.9639376218323586
Recall: 0.9629629629629629
F1 Score: 0.9628939284111699
```

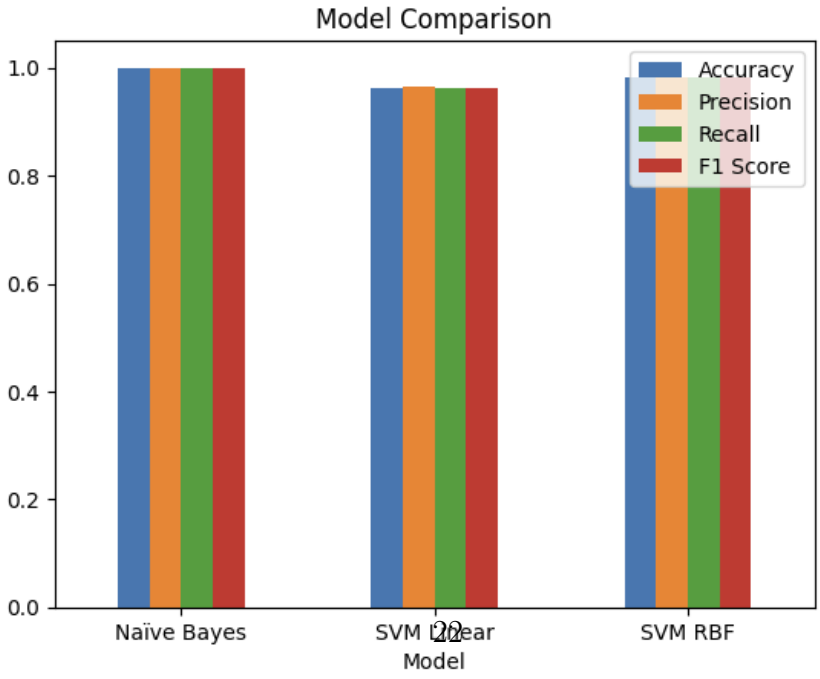



```
===== SVM (RBF Kernel) =====
Accuracy: 0.9814814814814815
Precision: 0.9823232323232324
Recall: 0.9814814814814815
F1 Score: 0.9813775282901185
```


```
===== SVM (RBF Kernel) =====
Accuracy: 0.9814814814814815
Precision: 0.9823232323232324
Recall: 0.9814814814814815
F1 Score: 0.9813775282901185
```


```
===== Model Comparison =====
         Model  Accuracy  Precision    Recall  F1 Score
0  Naïve Bayes  1.000000   1.000000  1.000000  1.000000
1   SVM Linear  0.962963   0.963938  0.962963  0.962894
2      SVM RBF  0.981481   0.982323  0.981481  0.981378
```



Experiment 7: Q-Learning in Grid Environment

Objective: To implement Q-learning in a grid-based environment and analyze how an agent learns an optimal policy using reinforcement learning with -greedy exploration.

Code:

```
# ================================
# 1. Import Libraries
# ================================
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import pandas as pd
import random

# ================================
# 2. Environment Setup
# ================================
GRID_SIZE = 4
START = (0, 0)
GOAL = (3, 3)
OBSTACLES = [(1, 1), (2, 1)]

ACTIONS = ['UP', 'DOWN', 'LEFT', 'RIGHT']
ACTION_MAP = {
    0: (-1, 0),  # UP
    1: (1, 0),   # DOWN
    2: (0, -1),  # LEFT
    3: (0, 1)    # RIGHT
}

# ================================
# 3. State Mapping
# ================================
def state_to_index(state):
    return state[0] * GRID_SIZE + state[1]

def index_to_state(index):
    return (index // GRID_SIZE, index % GRID_SIZE)

NUM_STATES = GRID_SIZE * GRID_SIZE
NUM_ACTIONS = len(ACTIONS)

# ================================
# 4. Initialize Q-table
# ================================
Q = np.zeros((NUM_STATES, NUM_ACTIONS))

# ================================
# 5. Hyperparameters
# ================================
alpha = 0.1       # learning rate
gamma = 0.9       # discount factor
epsilon = 1.0     # exploration rate
epsilon_decay = 0.995
epsilon_min = 0.01

episodes = 800

# ================================
# 6. Environment Step Function
# ================================
def step(state, action):
    move = ACTION_MAP[action]
    next_state = (state[0] + move[0], state[1] + move[1])

    # boundary check
    if next_state[0] < 0 or next_state[0] >= GRID_SIZE or next_state[1] < 0 or next_state[1] >=
        GRID_SIZE:
        next_state = state

    # obstacle
    if next_state in OBSTACLES:
        return next_state, -10, True
```

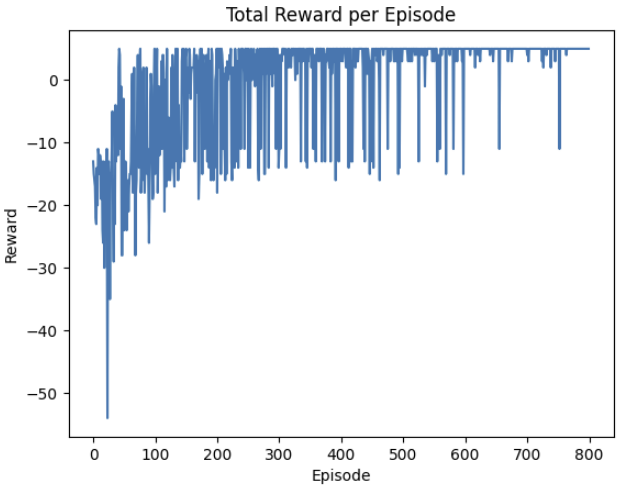
```
    # goal
    if next_state == GOAL:
        return next_state, 10, True

    return next_state, -1, False

# ================================
# 7.   -greedy Policy
# ================================
def choose_action(state_idx):
    if random.uniform(0, 1) < epsilon:
        return random.randint(0, NUM_ACTIONS - 1)
    return np.argmax(Q[state_idx])

# ================================
# 8. Training Loop
# ================================
rewards_per_episode = []

for ep in range(episodes):
    state = START
    state_idx = state_to_index(state)
    total_reward = 0
    done = False

    while not done:
        action = choose_action(state_idx)
        next_state, reward, done = step(state, action)

        next_state_idx = state_to_index(next_state)

        # Q-Learning Update Rule
        Q[state_idx][action] += alpha * (
            reward + gamma * np.max(Q[next_state_idx]) - Q[state_idx][action]
        )

        state = next_state
        state_idx = next_state_idx
        total_reward += reward

    rewards_per_episode.append(total_reward)

    # decay epsilon
    global epsilon
    if epsilon > epsilon_min:
        epsilon *= epsilon_decay

# ================================
# 9. Plot Reward vs Episodes
# ================================
plt.plot(rewards_per_episode)
plt.title("Total Reward per Episode")
plt.xlabel("Episode")
plt.ylabel("Reward")
plt.show()

# ================================
# 10. Extract Optimal Path
# ================================
def get_optimal_path():
    state = START
    path = [state]
    visited = set()

    while state != GOAL:
        state_idx = state_to_index(state)
        action = np.argmax(Q[state_idx])
        move = ACTION_MAP[action]

        next_state = (state[0] + move[0], state[1] + move[1])

        if next_state in visited:
            break  # avoid infinite loops

        visited.add(state)
        path.append(next_state)
        state = next_state
```

```
    return path

optimal_path = get_optimal_path()
print("\nOptimal Path:")
print(optimal_path)

# ================================
# 11. Policy Visualization
# ================================
policy_grid = np.empty((GRID_SIZE, GRID_SIZE), dtype=object)

for i in range(GRID_SIZE):
    for j in range(GRID_SIZE):
        state = (i, j)

        if state in OBSTACLES:
            policy_grid[i][j] = "X"
        elif state == GOAL:
            policy_grid[i][j] = "G"
        else:
            idx = state_to_index(state)
            best_action = np.argmax(Q[idx])
            policy_grid[i][j] = ACTIONS[best_action][0]  # first letter

print("\nPolicy Grid:")
print(policy_grid)

# Heatmap style visualization
policy_numeric = np.zeros((GRID_SIZE, GRID_SIZE))

for i in range(GRID_SIZE):
    for j in range(GRID_SIZE):
        if (i, j) in OBSTACLES:
            policy_numeric[i][j] = -1
        else:
            idx = state_to_index((i, j))
            policy_numeric[i][j] = np.max(Q[idx])

sns.heatmap(policy_numeric, annot=True)
plt.title("State Value Heatmap")
plt.show()

# ================================
# 12. Q-table Visualization
# ================================
plt.figure(figsize=(10, 6))
sns.heatmap(Q, annot=False)
plt.title("Q-table Heatmap")
plt.xlabel("Actions")
plt.ylabel("States")
plt.show()

# ================================
# 13. Analysis
# ================================
print("\nAnalysis:")
print("- Initially, rewards are low due to random exploration.")
print("- As training progresses, rewards increase as agent learns optimal path.")
print("-   -greedy balances exploration and exploitation.")
print("- Final policy directs agent toward goal while avoiding obstacles.")
print("- Q-values converge indicating stable learning.")
```

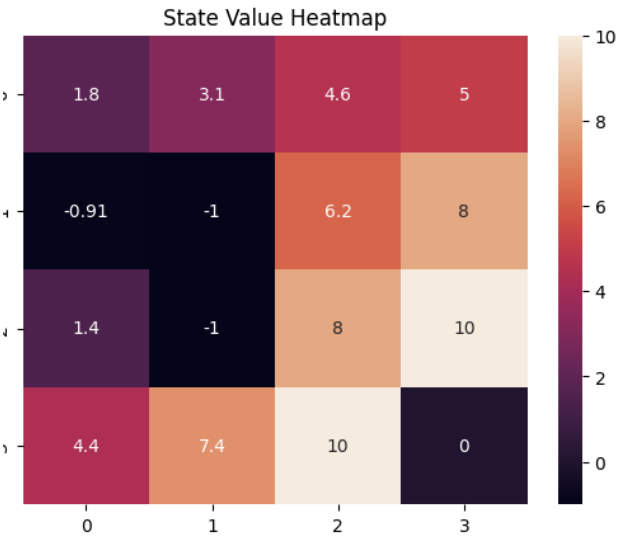
Conclusion:

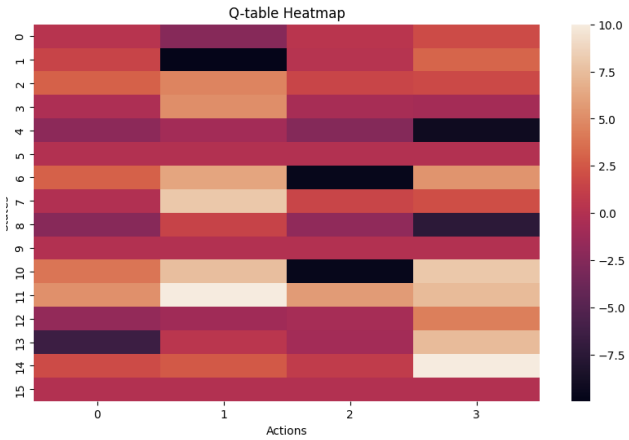
The experiment demonstrated Q-learning in a grid environment where the agent learns an optimal path through trial and error. The reward curve showed improvement over episodes, and the learned policy successfully guided the agent to the goal while avoiding obstacles.

Output:




```
Optimal Path:
[(0, 0), (0, 1), (0, 2), (1, 2), (2, 2), (2, 3), (3, 3)]

Policy Grid:
[['R' 'R' 'D' 'D']
 ['D' 'X' 'D' 'D']
 ['D' 'X' 'R' 'D']
 ['R' 'R' 'R' 'G']]
```






```
Analysis:
- Initially, rewards are low due to random exploration.
- As training progresses, rewards increase as agent learns optimal path.
- ε-greedy balances exploration and exploitation.
- Final policy directs agent toward goal while avoiding obstacles.
- Q-values converge indicating stable learning.
```

Figure 7: Outputs of Experiment 7

Experiment 8: Perceptron vs MLP (Neural Networks)

Objective: To compare the performance of a Perceptron and a Multi-Layer Perceptron (MLP) on a non-linear dataset, and analyze their ability to learn complex decision boundaries.

Code:

```
# ================================
# 1. Import Libraries
# ================================
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.datasets import make_moons
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import Perceptron
from sklearn.neural_network import MLPClassifier
from sklearn.metrics import (
    accuracy_score, precision_score, recall_score, f1_score,
    confusion_matrix, classification_report
)

# ================================
# 2. Load Dataset (Non-linear)
# ================================
# Using make_moons to show perceptron limitation
X, y = make_moons(n_samples=500, noise=0.2, random_state=42)

# ================================
# 3. Train-Test Split
# ================================
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# ================================
# 4. Feature Scaling
# ================================
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# ================================
# 5. Perceptron Model (Library)
# ================================
perc_model = Perceptron(max_iter=1000)
perc_model.fit(X_train, y_train)

y_pred_perc = perc_model.predict(X_test)

# ================================
# 6. Evaluation (Perceptron)
# ================================
print("\n===== Perceptron Results =====")

acc_perc = accuracy_score(y_test, y_pred_perc)
print("Accuracy:", acc_perc)

cm_perc = confusion_matrix(y_test, y_pred_perc)
print("\nConfusion Matrix:\n", cm_perc)

sns.heatmap(cm_perc, annot=True, fmt='d')
plt.title("Confusion Matrix - Perceptron")
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.show()

# ================================
# 7. MLP Model
# ================================
mlp_model = MLPClassifier(
    hidden_layer_sizes=(16,),
    max_iter=300,
```

```
    learning_rate_init=0.01,
    random_state=42
)

mlp_model.fit(X_train, y_train)

y_pred_mlp = mlp_model.predict(X_test)

# ================================
# 8. Evaluation (MLP)
# ================================
print("\n===== MLP Results =====")

acc_mlp = accuracy_score(y_test, y_pred_mlp)
prec_mlp = precision_score(y_test, y_pred_mlp)
rec_mlp = recall_score(y_test, y_pred_mlp)
f1_mlp = f1_score(y_test, y_pred_mlp)

print("Accuracy:", acc_mlp)
print("Precision:", prec_mlp)
print("Recall:", rec_mlp)
print("F1 Score:", f1_mlp)

cm_mlp = confusion_matrix(y_test, y_pred_mlp)
print("\nConfusion Matrix:\n", cm_mlp)

print("\nClassification Report:\n", classification_report(y_test, y_pred_mlp))

sns.heatmap(cm_mlp, annot=True, fmt='d')
plt.title("Confusion Matrix - MLP")
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.show()

# ================================
# 9. Loss Curve Visualization
# ================================
plt.plot(mlp_model.loss_curve_)
plt.title("MLP Loss Curve")
plt.xlabel("Epochs")
plt.ylabel("Loss")
plt.show()

# ================================
# 10. Model Comparison
# ================================
comparison = pd.DataFrame({
    "Model": ["Perceptron", "MLP"],
    "Accuracy": [acc_perc, acc_mlp],
    "Precision": [np.nan, prec_mlp],
    "Recall": [np.nan, rec_mlp],
    "F1 Score": [np.nan, f1_mlp]
})

print("\n===== Model Comparison =====")
print(comparison)

comparison.set_index("Model").plot(kind='bar')
plt.title("Perceptron vs MLP Comparison")
plt.xticks(rotation=0)
plt.show()

# ================================
# 11. Decision Boundary Visualization
# ================================
def plot_decision_boundary(model, X, y, title):
    x_min, x_max = X[:, 0].min() - 1, X[:, 0].max() + 1
    y_min, y_max = X[:, 1].min() - 1, X[:, 1].max() + 1

    xx, yy = np.meshgrid(
        np.linspace(x_min, x_max, 200),
        np.linspace(y_min, y_max, 200)
    )

    Z = model.predict(np.c_[xx.ravel(), yy.ravel()])
    Z = Z.reshape(xx.shape)

    plt.contourf(xx, yy, Z, alpha=0.3)
```

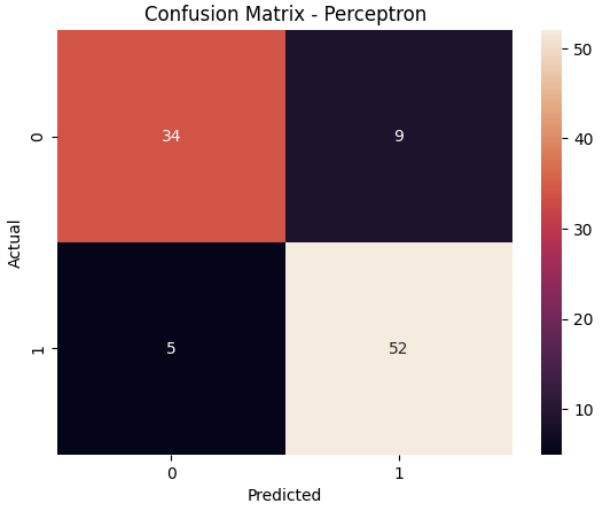
```
    plt.scatter(X[:, 0], X[:, 1], c=y)
    plt.title(title)
    plt.show()

plot_decision_boundary(perc_model, X_test, y_test, "Perceptron Decision Boundary")
plot_decision_boundary(mlp_model, X_test, y_test, "MLP Decision Boundary")

# ================================
# 12. Analysis
# ================================
print("\n===== Analysis =====")

print("1. Perceptron Limitations:")
print("- Can only classify linearly separable data.")
print("- Performs poorly on non-linear datasets like make_moons.")

print("\n2. MLP Advantages:")
print("- Hidden layer allows learning non-linear patterns.")
print("- Uses activation functions to create complex boundaries.")
print("- Achieves significantly better performance.")

print("\n3. Conclusion:")
print("- Perceptron is simple but limited.")
print("- MLP is more powerful and suitable for real-world problems.")
```

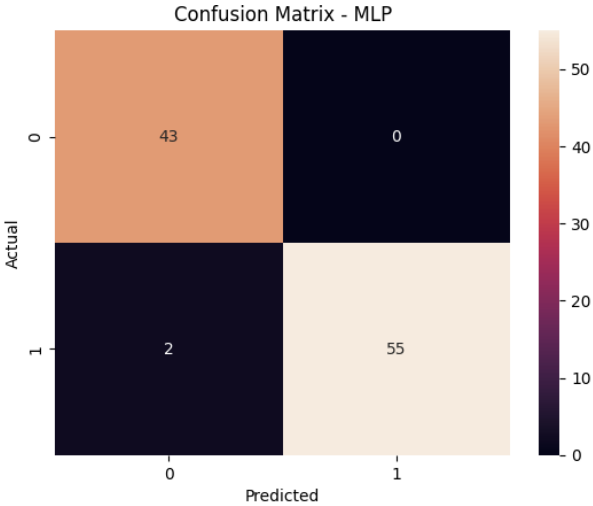
Conclusion:

The experiment showed that the Perceptron performs poorly on non-linear data due to its linear nature. In contrast, the MLP model successfully learned complex patterns using hidden layers and achieved better performance. This highlights the importance of neural networks for solving non-linear problems.

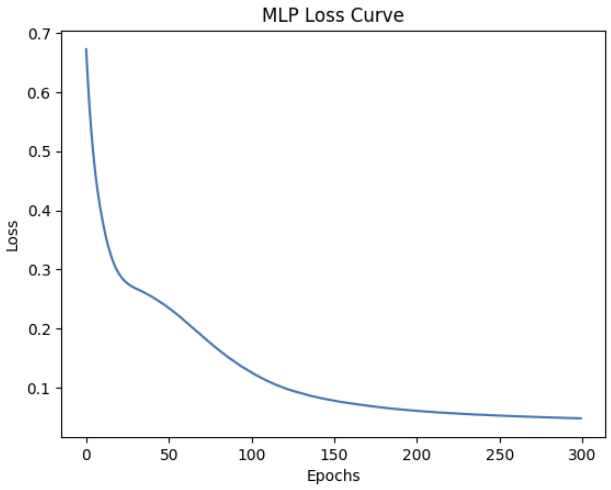
Output:

```
===== Perceptron Results =====
Accuracy: 0.86

Confusion Matrix:
 [[34  9]
 [ 5 52]]
```






```
===== Model Comparison =====
        Model  Accuracy  Precision    Recall  F1 Score
0  Perceptron      0.86        NaN       NaN       NaN
1         MLP      0.98        1.0  0.964912  0.982143
```



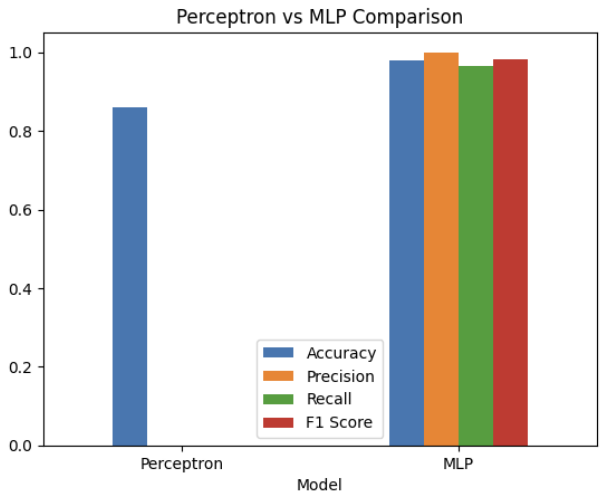


Figure 8: Outputs of Experiment 8 (Part 1)

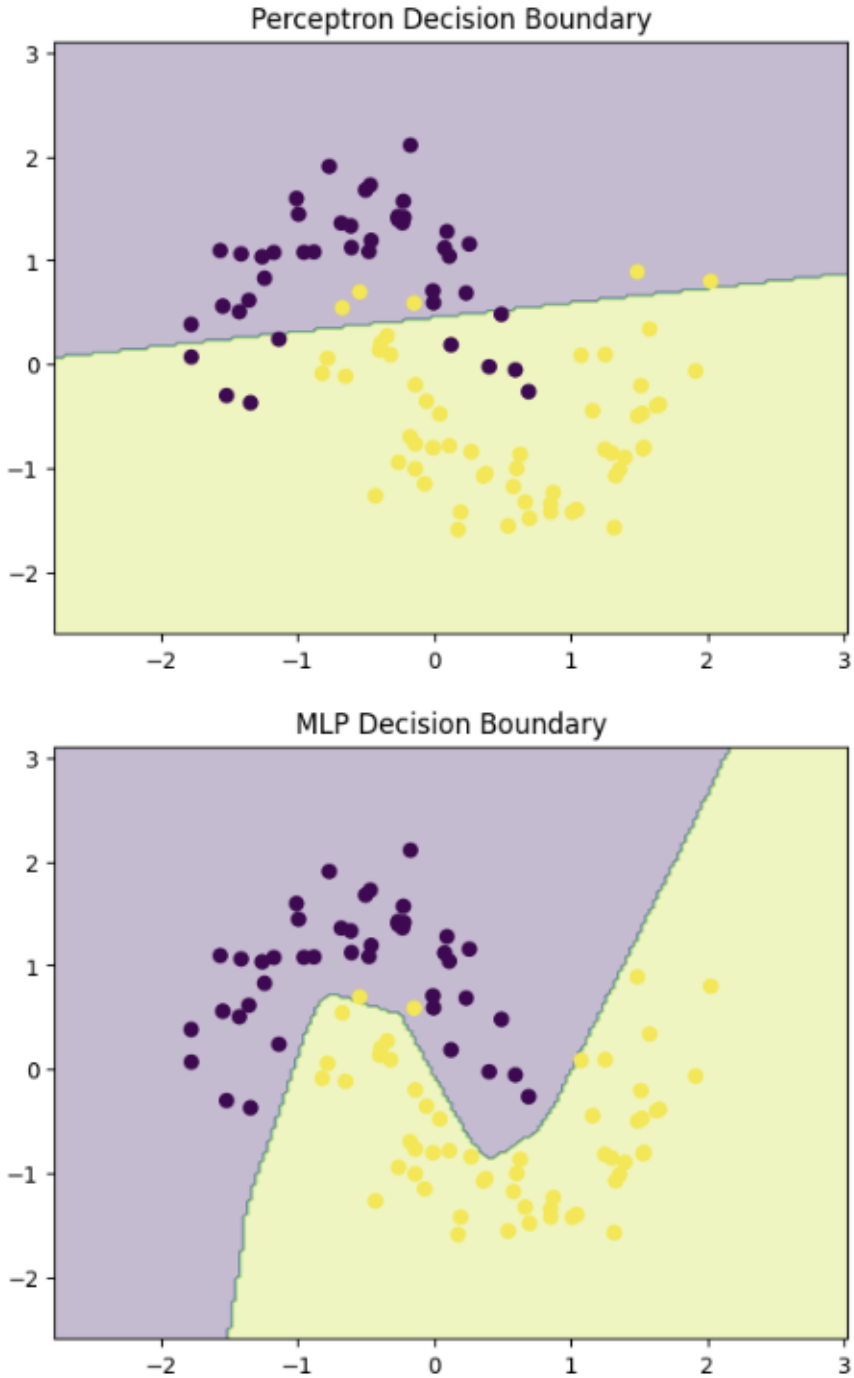


Figure 9: Outputs of Experiment 8